COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Oct 20, 2026	**DATE SUBMITTED:**	Oct 20, 2026
NAME:	Isaiah Benedict R. Garbosa Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14.1 Algorithm Complexity

1. PROCEDURE OUTPUT

	Best Case	Worst Case	Analysis
Algorithm 1: Quick Sort	O(n log n)	$O(n^2)$	The best case for quick sort is O(n log n) since the pivot splits the array into two nearly equivalent halves. While its worst is $O(n^2)$ since the selected pivot can constantly be the smallest or largest element in the array, which the algorithm fails to divide the array evenly.
Algorithm 2: Bubble Sort	O(n)	$O(n^2)$	The best case for quick sort is O(n) since the array can be already sorted, thus the for loop runs once. The worst case is $O(n^2)$ since sometimes, the algorithm can run a full iteration of both for loops. This happens if the array is in complete reverse order.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Algorithm 1: Quick Sort	Algorithm 2: Bubble Sort
Best Case	BEST CASE = 10 Comparisons: 27 Swaps: 15 Total Operations: 42 Runtime: 0.0004 ms BEST CASE = 1000 Comparisons: 10449 Swaps: 5797 Total Operations: 16246 Runtime: 0.0655 ms	BEST CASE = 10 Comparisons: 45 Swaps: 0 Total Operations: 45 Runtime: 0.0002 ms BEST CASE = 1000 Comparisons: 499500 Swaps: 0 Total Operations: 499500 Runtime: 0.6662 ms

	BEST CASE = 10000 Comparisons: 157696 Swaps: 86270 Total Operations: 243966 Runtime: 0.8134 ms	BEST CASE = 10000 Comparisons: 49995000 Swaps: 0 Total Operations: 49995000 Runtime: 46.2122 ms
Worst Case	WORST CASE = 10 Comparisons: 45 Swaps: 29 Total Operations: 74 Runtime: 0.0005 ms WORST CASE = 1000 Comparisons: 499500 Swaps: 250499 Total Operations: 749999 Runtime: 1.2052 ms WORST CASE = 10000 Comparisons: 49995000 Swaps: 25004999 Total Operations: 74999999 Runtime: 139.333 ms	WORST CASE = 10 Comparisons: 45 Swaps: 45 Total Operations: 90 Runtime: 0.0004 ms WORST CASE = 1000 Comparisons: 499500 Swaps: 24093 Total Operations: 523593 Runtime: 0.7663 ms WORST CASE = 10000 Comparisons: 49995000 Swaps: 245616 Total Operations: 50240616 Runtime: 47.9471 ms

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Contrast and compare the two searching techniques used in class. The required deliverables for this activity would be the theoretical and empirical

	Binary Search	Linear Search
Pseudocode	START INITIALIZE low :=0, up=n-1 REPEAT WHILE low <= up mid = int(low+up)/2 IF(no=arr[mid]) PRINT "Search element is found!"	START N -> Boundary of the list Item -> Searching number Data -> Linear array INITIALIZE I = 0 REPEAT WHILE I <= n IF (item = data[i])

	EXIT ELSE IF(no < arr[mid]) then up = mid-1 ELSE low = mid+1 PRINT "Search element is not found" END	PRINT "Searching is successful" EXIT ELSE PRINT "Searching is Unsuccessful" EXIT END
Analysis	The binary search uses the sorted data to its advantage as it finds the middle and compares whether the data matches the search element. If it matches the current middle element then the search element is found. If the search element is found to be less than the middle element, then it searches the lower half using its lower middle element. Then if the search element is found to be higher than the middle element then it searches the upper half. It does this until the search element is found or it has searched through the entire list.	The linear search traverses through the list one-by-one until it finds the search element. This works with an unsorted array as every element is not sorted. While it works for sorted arrays, compared to binary search, it's inefficient.
Time Complexity	T(n) = log n	T(n) = n
Space Complexity	(N * 4) + 16 bytes	(4 * N) + 16 bytes

Input Size (N)	Elapsed TIme for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	Binary Search: Searching is successful! Found at index: 999 Execution Time: 335 microseconds	Linear Search: Searching is successful! Found at index 999 Execution Time: 420 microseconds
10,000	Binary Search: Searching is successful! Found at index: 9999 Execution Time: 347 microseconds	Linear Search: Searching is successful! Found at index 9999 Execution Time: 506 microseconds
100,000	Binary Search: Searching is successful! Found at index: 99999 Execution Time: 497 microseconds	Linear Search: Searching is successful! Found at index 99999 Execution Time: 659 microseconds

Analysis

Comparing the two search algorithms, binary search proves to be the fastest according to theoretical and empirical analysis. From the theoretical analysis, the time complexity for binary search is log n, while linear search is n. Log n is faster than n as it cuts the process in half. This is also supported by how the binary search works from its pseudocode. It accesses the middle element, compares, and makes decisions from the comparison. On the other hand, linear search becomes more inefficient as it traverses the list one at a time. The empirical analysis also supports this as the binary search is still the fastest among the two, even as the input size grows.

Conclusion - Garbosa

In this activity, I learned more about algorithm complexity, more specifically, time complexity and space complexity. I was able to compare different algorithms from our past work to see which algorithms are most efficient and which are less efficient. We were able to notice that results using theoretical tools and experimental tools mostly match. However, it can still depend on the hardware of the computer that the user is using since some computers can run the program faster while some can run slower. Furthermore, I noticed that runtimes can vary based on the value of the size of an array. The higher the value gets, the more it takes to process. Also, we noticed that different algorithms consume different sizes of bytes based on how many variables are used. We also noticed that the amount of bytes consumed by the program does not directly affect the runtime. So, some algorithms can consume the same amount of space but have a different time complexity. Overall, I found the activity helpful for future projects as understanding time complexity and space complexity can be used to choose and compare different algorithms in order to choose one that is efficient and most suited for a project. I believe that I was able to further understand the importance of time complexity and space complexity and would be able to use them properly for future projects.

Conclusion - Vizcayno

For this activity, I learned how to compare certain algorithms together using theoretical and empirical analysis. When it comes to finding an efficient algorithm to solve certain problems, I could use theoretical analysis as a guide as to how its outputs can look compared to other algorithms. Although, if these algorithms were to be run on different devices, its outputs can vary and this is called empirical analysis. Since devices vary from one to another, its better to use experimental tools. To summarize, if we want to find the most optimized and efficient algorithm, it is wise to use both theoretical and empirical analysis since theoretical is machine independent and empirical is machine dependent. You cannot exactly just choose one, as both are needed to create an analysis covering perspectives that will help you determine the algorithms' strengths and weaknesses. At the end of the activity, I find myself wondering about other comparisons with other algorithms. I think we did well in this activity as it was easy to do and we were able to complete it within the given time limit. When it comes to improvements, maybe I could explore more options or expand my knowledge and understanding of theoretical and empirical analysis.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.